LAPORAN PRAKTIKUM BIG DATA

Pemrosesan Terdistribusi PySpark RDD

Dataset: Online Retail II Dataset (Kaggle/UCI) | **Lingkungan:** PySpark (Databricks / Colab)

Ringkasan Eksekutif: Laporan ini mendokumentasikan implementasi pemrosesan Big Data menggunakan *Resilient Distributed Dataset* (RDD) pada ekosistem Apache Spark (PySpark). Analisis mencakup tahap ekstraksi data, inisialisasi lingkungan terdistribusi, pembersihan data, serta eksekusi serangkaian algoritma MapReduce untuk mendapatkan *Business Intelligence* (BI) dan mengukur metrik *Performance Engineering*.

BAGIAN I: PERSIAPAN LINGKUNGAN & DATA

1. Pengambilan Dataset ke Sistem (DBFS / Colab)

Dataset diunduh langsung dari repositori publik (UCI/Kaggle API) ke penyimpanan lokal node (atau DBFS) untuk menghindari proses unggah manual yang tidak efisien pada data bervolume besar.

```
# Menggunakan Kaggle CLI untuk mengunduh dan mengekstrak dataset import os
os.environ['KAGGLE_USERNAME'] = "USERNAME_KAGGLE_ANDA" os.environ['KAGGLE_KEY'] =
"KEY_KAGGLE_ANDA" %sh # Unduh dataset langsung dari Kaggle dan ekstrak ke /tmp kaggle
datasets download -d vijayuv/onlineretail -p /tmp/ --unzip cp /tmp/*.csv
/dbfs/FileStore/tables/online_retail.csv
```

2. Inisialisasi SparkContext

Membangun sesi Spark lokal dengan mengalokasikan seluruh *core* komputasi yang tersedia (`local[*]`) untuk memaksimalkan paralelisasi.

```
from pyspark.sql import SparkSession spark = SparkSession.builder \
.appName("Praktikum_OnlineRetail") \ .master("local[*]") \ .getOrCreate() sc =
spark.sparkContext print("--- INFORMASI LOCAL SPARK ---") print(f"Versi PySpark :
{spark.version}") print(f"Master Node : {sc.master}")
```

3. Membaca Dataset & Konversi RDD

Dataset dibaca sebagai DataFrame, dibersihkan dari nilai *Null* pada kolom vital, kemudian dikonversi menjadi struktur dasar *Tuple* RDD berisi 8 elemen untuk diproses menggunakan API tingkat rendah Spark, dan dipartisi ulang.

```
# Membaca dataset CSV ke DataFrame dan menghapus baris bernilai NULL df_raw = spark.read.format("csv").option("header", "true").load(file_path) df_clean = df_raw.dropna(subset=["InvoiceNo", "CustomerID", "UnitPrice", "Quantity"]) # Konversi ke RDD Tuple dan Partisi rdd_raw = df_clean.rdd.map(lambda row: ( str(row['InvoiceNo']), str(row['StockCode']), str(row['Description']), int(row['Quantity']), float(row['UnitPrice']), str(row['CustomerID']), str(row['Country']), str(row['InvoiceDate']) )) rdd = rdd_raw.repartition(8) print(f"Jumlah Total Record Terbersih: {rdd.count():,}")
```

```
CONSOLE OUTPUT
Jumlah Total Record Terbersih: 417,534 Contoh 3 Transaksi Pertama: ('489438', '21411', 'GINGHAM HEART DOORSTOP RED', 32, 2.5, '18102.0', 'United Kingdom', '2009-12-01 09:24:00') ('489438', '84031A', 'CHARLIE+LOLA RED HOT WATER BOTTLE ', 56, 3.0, '18102.0', 'United Kingdom', '2009-12-01 09:24:00') ('489438', '84031B', 'CHARLIE LOLA BLUE HOT WATER BOTTLE ', 56, 3.0, '18102.0', 'United Kingdom', '2009-12-01 09:24:00')
```

BAGIAN II: TRANSFORMASI & AGREGASI DASAR

4. Transformasi RDD (Filter & Map)

Tahap *data wrangling* untuk mengekstraksi hanya transaksi domestik yang valid (kuantitas dan harga lebih dari 0) dan memetakan struktur data menjadi pasangan kunci-nilai (*Key-Value Pair*) berbasis Nomor Invoice dan Nilai Belanja.

```
# Filter transaksi valid dari UK (Qty > 0 & Price > 0) rdd_uk_valid = rdd.filter(lambda x: x[3] > 0 and x[4] > 0 and x[6] == 'United Kingdom') # Buat Pair RDD (Invoice, TotalSpent) rdd_invoice_spent = rdd_uk_valid.map(lambda x: (x[0], round(x[3] * x[4], 2)))
```

```
CONSOLE OUTPUT
Jumlah Transaksi Valid UK: 370,929 Contoh 3 Pair RDD: [('489438', 80.0), ('489438', 168.0), ('489438', 168.0)]
```

5. Agregasi RDD (reduceByKey & Sorting)

```
# Jumlahkan total belanja per invoice & Urutkan descending
rdd_total_per_invoice = rdd_invoice_spent.reduceByKey(lambda a, b: a + b)
rdd_sorted_invoice = rdd_total_per_invoice.sortBy(lambda x: x[1], ascending=False)
top_10_invoices = rdd_sorted_invoice.take(10)
```

```
CONSOLE OUTPUT
Waktu Eksekusi Agregasi RDD: 5.0038 detik --- TOP 10 INVOICE DENGAN TRANSAKSI TERBESAR
--- Invoice ID: 524181 | Total Belanja: £ 33,167.80 Invoice ID: 526934 | Total
Belanja: £ 26,007.08 Invoice ID: 515944 | Total Belanja: £ 22,863.36
```

Analisis: Fungsi `reduceByKey` berhasil mengagregasi 370 ribu *record* dalam waktu sangat efisien (~5 detik). Ditemukan Invoice tunggal terbesar senilai lebih dari £33.000.

6. Optimasi RDD (Caching & Persistence)

```
from pyspark import StorageLevel # Simpan RDD ke RAM
rdd_sorted_invoice.persist(StorageLevel.MEMORY_ONLY) # Mengukur pembacaan data sebelum
vs sesudah Cache (di memori) t0 = time.time(); count_1 = rdd_sorted_invoice.count();
t1 = time.time() - t0 t2 = time.time(); count_2 = rdd_sorted_invoice.count(); t3 =
time.time() - t2
```

```
CONSOLE OUTPUT
Eksekusi Aksi Pertama (Tanpa Cache Warmup) : 0.7174 detik Eksekusi Aksi Kedua
(Menggunakan Cache) : 0.3896 detik
```

Analisis: Eksekusi tahap kedua menunjukan akselerasi hampir 50%. Mekanisme `persist(MEMORY_ONLY)` terbukti vital mengurangi latensi I/O sistem pada RDD yang diakses berulang.

7. Analisis Produk Paling Laris

```
# Pair RDD: (Description, Quantity) rdd_product_qty = rdd_uk_valid.map(lambda
x: (x[2], x[3])) \ .reduceByKey(lambda a, b: a + b) \ .sortBy(lambda x: x[1],
ascending=False)
```

```
CONSOLE OUTPUT
--- TOP 5 PRODUK TERLARIS --- Produk: WHITE HANGING HEART T-LIGHT HOLDER | Jumlah Terjual: 51,679 unit Produk: WORLD WAR 2 GLIDERS ASSTD DESIGNS | Jumlah Terjual: 51,538 unit
```

BAGIAN III: ANALISIS DATA & EKSPLORASI (KATEGORI A)

Tugas A1: Analisis Pelanggan Utama Luar Negeri (Non-UK)

```
# 1. Filter Non-UK dan validitas rdd_non_uk = rdd.filter(lambda x: x[6] != 'United Kingdom' and x[3] > 0 and x[4] > 0) # 2 & 3. Map (CustomerID, Spent), Reduce, Sort top_5_foreign = rdd_non_uk.map(lambda x: (x[5], x[3] * x[4])) \ .reduceByKey(lambda a, b: a + b) \ .sortBy(lambda x: x[1], ascending=False).take(5)
```

```
CONSOLE OUTPUT
--- TOP 5 PELANGGAN LUAR NEGERI (NON-UK) --- CustomerID: 14646.0 | Total Pengeluaran: £ 248,396.50 CustomerID: 14156.0 | Total Pengeluaran: £ 196,566.74
```

Tugas A2: Analisis Potensi Kerugian Transaksi Pembatalan

```
# 1. Filter transaksi pembatalan (Invoice berawalan 'C' / Qty < 0) rdd_canceled = rdd.filter(lambda x: str(x[0]).upper().startswith('C') or x[3] < 0) # 2 & 3. Hitung jumlah dan estimasi kerugian absolut total_cancel_count = rdd_canceled.count() total_loss = rdd_canceled.map(lambda x: abs(x[3]) * x[4]).sum()
```

```
CONSOLE OUTPUT
--- TUGAS A2: ANALISIS POTENSI KERUGIAN --- Jumlah Transaksi Dibatalkan : 9,839 transaksi Total Estimasi Kerugian : £ 483,794.70
```

Analisis Kategori A: Eksplorasi bisnis mengungkap volume retur barang sangat masif mendekati angka 10 ribu transaksi, memicu kerugian/*refund* finansial hampir £500 ribu. Metrik ini krusial untuk departemen logistik. Di sisi lain, pembelanja lintas negara (contoh ID: 14646.0) memberikan kontribusi kapital yang sangat signifikan.

BAGIAN IV: OPTIMASI PERFORMA & BENCHMARK (KATEGORI B)

Tugas B1: Eksperimen Dampak Distribusi Partisi

```
for p in [2, 8, 16]: start_time = time.time() # repartition() mendistribusikan data, count() memicu Lazy Evaluation
rdd_invoice_spent.repartition(p).reduceByKey(lambda a, b: a + b).count() print(f"Waktu Eksekusi ({p:2} Partisi) : {time.time() - start_time:.4f} detik")
```

```
CONSOLE OUTPUT
--- TUGAS B1: BENCHMARK PARTISI --- Waktu Eksekusi ( 2 Partisi) : 3.8120 detik Waktu Eksekusi ( 8 Partisi) : 3.1740 detik Waktu Eksekusi (16 Partisi) : 3.4637 detik
```

Tugas B2: Perbandingan reduceByKey vs groupByKey

```
# reduceByKey() start_rbk = time.time() rdd_desc_qty.reduceByKey(lambda a, b: a + b).count() time_rbk = time.time() - start_rbk # groupByKey() start_gbk = time.time() rdd_desc_qty.groupByKey().mapValues(sum).count() time_gbk = time.time() - start_gbk
```

```
CONSOLE OUTPUT
--- TUGAS B2: reduceByKey vs groupByKey --- Waktu eksekusi reduceByKey() : 1.7810 detik Waktu eksekusi groupByKey() : 1.5339 detik Selisih waktu eksekusi : 0.2471 detik
```

Analisis Kategori B (Performance Engineering): Eksperimen partisi membuktikan konsep keseimbangan paralelisasi Spark: 8 partisi menorehkan waktu tercepat, selaras dengan jumlah komputasi optimal untuk ukuran data ini. Terlalu sedikit partisi (2) mempersempit jalur data, sedangkan terlalu banyak (16) memperberat sistem dengan *overhead* manajemen. Secara umum, secara teoretis `reduceByKey` lebih disarankan pada infrastruktur *multi-node cluster* dibanding `groupByKey` karena keberadaan mekanisme agregasi lokal (*map-side combine*) meminimalkan *network shuffle* massal yang memicu OOM (Out-of-Memory). Fluktuasi hasil pada percobaan B2 dipengaruhi kondisi mesin virtual *single-node* (Colab).

BAGIAN V: PEMROSESAN STRING, WAKTU, & HIMPUNAN (KATEGORI C & D)

Tugas C1: Analisis Jam Sibuk (Peak Transaction Hours)

```
# Memecah string waktu (YYYY-MM-DD HH:MM:SS) untuk ekstraksi Jam (HH) rdd_hour = rdd.map(lambda x: (str(x[7]).split(' ')[1].split(':')[0], 1)) top_3_hours = rdd_hour.reduceByKey(lambda a, b: a + b) \ .sortBy(lambda x: x[1], ascending=False).take(3)
```

CONSOLE OUTPUT

```
--- TUGAS C1: TOP 3 JAM SIBUK TRANSAKSI --- Pukul 12:00 | Jumlah Transaksi: 73,677 Pukul 13:00 | Jumlah Transaksi: 68,900 Pukul 14:00 | Jumlah Transaksi: 57,281
```

Tugas C2: Kalkulasi Rata-rata Kompleks (Produk "HEART")

```
# 1. Filter deskripsi mengandung kata HEART rdd_heart = rdd_uk_valid.filter(lambda x: x[2] is not None and 'HEART' in str(x[2]).upper()) # 2. Pair: (Description, (Price, 1)) -> reduceByKey untuk sum & count akumulatif rdd_heart_pair = rdd_heart.map(lambda x: (x[2], (x[4], 1))) rdd_heart_avg = rdd_heart_pair.reduceByKey(lambda a, b: (a[0] + b[0], a[1] + b[1])) \ .map(lambda x: (x[0], round(x[1][0] / x[1][1], 2)))
```

CONSOLE OUTPUT

```
--- TUGAS C2: RATA-RATA HARGA PRODUK 'HEART' --- Total Varian Produk Ditemukan: 265 varian Contoh Produk: CREAM SWEETHEART MINI CHEST | Rata-rata Harga: £ 12.67
```

Tugas D1: Eksklusivitas Pasar (Operasi RDD subtract)

```
# Memanfaatkan operasi Set RDD subtract() rdd_germany = rdd.filter(lambda x: x[6] == 'Germany' and x[3] > 0).map(lambda x: x[2]).distinct() rdd_france = rdd.filter(lambda x: x[6] == 'France' and x[3] > 0).map(lambda x: x[2]).distinct() rdd_germany_only = rdd_germany.subtract(rdd_france)
```

```
CONSOLE OUTPUT
--- TUGAS D1: PRODUK EKSKLUSIF GERMANY (VS FRANCE) --- Jumlah Produk Eksklusif: 621
produk Contoh: JUMBO BAG SCANDINAVIAN PAISLEY, DOORMAT BLACK FLOCK
```

Analisis Kategori C & D: Transaksi mencapai puncaknya pada jam istirahat siang (Pukul 12:00). Pada analisis teks dan himpunan, PySpark terbukti andal dalam menyelesaikan ekstraksi nama string kompleks secara cepat. Ditemukan 621 jenis produk di pasar Jerman yang tak memiliki daya tarik (atau belum rilis) di Prancis berdasarkan histori belanja.